# Skuld

**Le Ol' CFD One Shotter**

---

Colin Gray, PARSEC, Caltech CCE

ChE090 Senior Thesis Proposal

Advisors:
Professor James Braun, NCSU (Primary)
TBD, Caltech (Advisor of Record)

**Abstract:**

Reading about Loci gave me the initial idea for this senior thesis: treating a simulation as a program that can be built from a set of rules [1]. Large language models (LLMs) are *capable* at coding but less reliable as physics authorities. Inspired by DreamCoder, AlphaEvolve [2,3], and discussions with colleagues at JHU APL, I believe a specific architecture would allow for "one-shotting" of new computational fluid dynamics (CFD) geometries, scales, chemistries, and phases that were not specifically trained on. Here, 'one-shotting' is a fun term meaning that, with only one prompt, an unseen (held-out) task can be completed without further human intervention. The problem, allowable actions, evidence plan, and budgets are specified before the run based on my prompt, and the scored outcome is fed back into the process until the task is finished. The "aspirational" final goal (Tier 5) would be one-shotting of a 2D reacting, unsteady, transient, liquid Jet A/gaseous-air rotating detonation engine [14]. With Professor James Braun's assistance, available experimental data will support the development and validation of lower-order cases, while exact or manufactured solutions, conservation checks, and observed-order tests will be used for verification. I do not have any raw data or publicly available high fidelity CFD data to compare to. Most of the lower-order targets will also lack direct data to compare against (without some lily-pad hops). This thesis therefore asks whether a one-prompt process of solver generation, monitoring, manipulating, and judging can produce useful CFD results and the limits. This is a chemical engineering senior thesis joining chemE, mechE, and applied AI.

**Disclaimer:**

To put it up front, the idea of an LLM running CFD, CADing, performing design optimization, and other engineering tasks is not a foreign concept. Related work already shows that LLMs can automate case setup, generate solver code for small PDE problems, modify source code, monitor simulations, and use runtime feedback [3-6]. I am not claiming to invent any one of those pieces. This is a paper tackling the challenging task of fast, autonomous, verifiable data generation of environments, conditions, and geometries that haven't been seen. The goal is to build something that can get us the best first guess in the shortest amount of time. This would never operate or be verifiable without a subject matter expert interpreting finished results. The goal is not to replace the engineer, but to reduce the time required to produce a defensible first-pass simulation.

**The Meat:**

Automating CFD workflows through LLMs is severely limited by the mismatch between an LLM's token-based reasoning and a CFD trajectory: a large, structured state which can span millions of cells evolving over space and (even worse) time. Modern models can produce and run a configuration for a given system or geometry and even satisfy a solver's convergence criteria. These simulations often don't converge, and convergence itself might be accompanied by incorrect equations, boundary problems, lack of closure, missing or duplicated source terms, bad mesh transfers, or unphysical numerical regimes [4,5]. I myself have seen the most beautiful rocket engine plumes, well within my convergence cutoff, secretly generate 200+ kg/s of mass flow. Similarly, cases of language models generating small solvers for canonical PDEs have been demonstrated [6], but said demonstrations do not build the numerical knowledge required for an unfamiliar multiphysics case. The numerical knowledge in this case is the operating envelope I will be providing to Skuld. These methods will carry their own failure models, observer hooks, cost, line of evidence, and rigor. The LLM will not be sifting through any past logs or evidence, and even while CFD is running, specific streamlined telemetry and injected history based on flags will replace the need for an LLM to monitor and process history itself.

This thesis proposes Skuld: an LLM sat inside a harness which will build *and* pilot the CFD itself. The harness is the actual thesis, but it's cooler to refer to the LLM as Skuld, the Norse mythological figure that interfaces with the future, i.e. Fate. For reference, there was a precursor named Surtr, the Norse fire giant, named thanks to a cool scene from God of War. The Norse names sort of… stuck? Harnesses expose "tools" to an LLM, which are operations it can take such as Edit, Write, Read, Bash, etc. Every time you speak to an LLM in a harness, such as Claude Code, the LLM has to read and choose which tool to use and how to use it. Recently, programmatic tool use, in which an LLM writes its own executable functions, has become more prevalent as they allow an LLM to swiftly solve its own problems. Skuld's recursive harness is similar to Prime Intellect's Prime Agent, except the IPython kernel is replaced with a CFD build environment [7]. Prime Agent is an RLM-native coding and research harness built around a persistent Python REPL, recursive subagents, and durable state; Skuld borrows that persistent-runtime pattern but rebuilds it around a CFD environment. The architectural inspiration for Skuld comes from a paper about Loci, which showed that you could independently author a simulation built from "rules," which would create a solver when different cases shared data and semantics. Skuld extends that idea across heterogeneous CFD ecosystems. It harvests numerical methods, tests, actual physical anchors, pathologies, and successful simulations from systems such as SU2, OpenFOAM, AMReX/Pele, Cantera, and SDT [8,9]. Where software cannot be reused directly, Skuld may write a narrow adapter, observer, test, or numerical function from published methods and documented behavior. These assets would be broken down and entered into a typed "Primitive Atlas" containing five categories of knowledge: ways to compute, ways to observe, ways CFD fails, ways to intervene, and ways to *know*. 'Ways to know' refers to a complex system of order checks, manufactured data, sensitivity studies, physical anchors, and an access-limited history of all past envelopes. One must imagine themselves being tasked with modeling a geometry they have never seen before, on a planet they know nothing about, and the only thing you're given is Python's official documentation.

For a new geometry and physical question, Skuld uses the Primitive Atlas to induce an Adaptive Solver Program (ASP, aka the CFD): governing equations, closures, mesh policies, numerical operators, backend, generated kernels/adapters/tests, live observers, allowable interventions, and an "evidence plan" stating what each requested output must show. It may seem redundant to propose clear conditions for a simulation before seeing it: put briefly, an LLM freshly injected with context from the Primitive Atlas is the best guess for clear conditions at any point in this system. After creating this CFD program, Skuld encounters the first check. The ASP must pass the Wall. The Wall is essentially a package of established verification and validation methods around the candidate program: type and unit checks, closure completeness, compilation, manufactured/exact solutions, observed order, conservation and property tests, controlled mutations, checkpoint/restart, donor comparisons where appropriate, and short mini simulations [10,11]. To my dismay, this will need an agentic component to compare the proposed solution to the original prompt. Without this, LLMs tend to rescue themselves from challenging obstacles by simply walking around it.

Now we get to the fun part. The ASP has passed the Wall, compiled, and is now running. During the run, Skuld acts like Dr. Strange. Hard numerical checks, pattern detectors, learned spatiotemporal observers, and independent shadow calculations watch the run and produce a compact incident when something looks wrong. Skuld does not continuously read the full CFD field, it is only called if an anomaly occurs in the first place. One frame of any simulation over 25,000 cells is already beyond the size of any frontier model's context. Once alerted, Skuld can see Cell-to-Code traces in the affected cells and faces down to the numerical operator and source code that produced them. I've chosen this Cell-to-Code method because it's easily verifiable and it would be cool. Skuld may then propose **one** typed action: rewind to a checkpoint, reduce timestep, refine the mesh, locally recompute with an operator [12], activate a larger/smaller domain, switch among prequalified (in the proposal phase) surrogate models, PDE solvers, request new evidence from a lower-order sim. I will also experiment with allowing an LLM to directly manipulate values in a cell. A deterministic Action Constitution verifies that the action preserves the intended problem, conservation laws, evidence requirements, and computational budget. Once again, there is a dreaded but required LLM component here. The change runs on a short checkpoint branch and is committed only if it fixes the named problem without breaking the rest of the case; otherwise, it rolls back. The exact permissions are research targets.

Now we come to the part of the thesis I dread. Make or break for the entire project. The Judge. All data generation comes down to how you can trust the output of something neither simulated nor sampled. An independent Judge will grow throughout this project from a library of invariant and evidence primitives. For an unprecedented case with no matching experiment, the harness decomposes the geometry into local regimes and asks which implementation tests, anchors, resolution studies, shadows, sensitivities, and known failure mechanisms support each requested quantity. It can't say "Yes this is correct." No one can. Supposedly, the data doesn't exist. It *can* say whether a claim is supported, bounded, unsupported, or rejected over the space and time being investigated. LLM to ensure prompt consistency once again. Ideally it would make it significantly harder for an incorrect simulation result to pass as correct, and would allow the tool to iterate until a justifiable solution or an honest stop. Every intervention remains part of

the claim and can lower confidence rather than disappearing from the record. This part of the project allows for extrapolation to unseen targets. Every successful solver or subgraph could become a reusable primitive only after held-out recurrence occurs. Otherwise, it isn't promoted.

**Goals and Demonstrations (Tier 3-5 are stretch goals):**

1. Tier 1 – (MINIMUM DEFENSIBLE SENIOR THESIS OUTCOME) Verification on a lower-order CFD. Given a prompt and a geometry, Skuld produces and executes a lower-order CFD. It compiles, runs, and matches the exact or manufactured solution. My first targets are a simple tube flow, air conditioning vents, aerofoil, and hopefully a reacting 2D liquid rocket engine plume sim. Would still be a very good outcome.
2. Tier 2 - Skuld detects a stable but nonphysical pressure anomaly, traces it to an AMR update issue, rewinds to a checkpoint, applies a prequalified non-AMR correction, and reaches the known final state without violating the ASP invariants.
3. Tier 3 - An unseen 3D film-cooling LES simulation returns a Judged and bounded cooling-effectiveness claim for a withheld hole-pattern for a given geometry and range of operating conditions.
4. Tier 4 – High-speed physics. The same architecture takes on thermochemistry, stiff source terms, AMR, detonation, and spray precursors, ending with a sustained gas-RDE mission and a Judge report [9,16]. Skuld will need to learn the beautiful world of surrogates.
5. Tier 5 – Finally, a simulation where every part of this needs to function over the course of many iterations on a computationally expensive prompt. A gas and liquid RDE simulation where Skuld tracks wave, injection, chemistry, and spray failure modes and the Judge will tell us how well the data applies to reality based on the path it took to get there [14-16].

**Methodology (Figure 1):**

- CFD Builder:
    - SolverGraph representing state, units, species, topology, operators, schedules, and (historical) observer hooks, legal interventions, evidence obligations. Evidence obligations are things that must be physically shown to prove something else. So, each requested quantity ("External flow, high pressure, reacting, steady state, etc.") is a claim + evidence row filled with historical required checks, acceptance criteria, commons failures modes, stop conditions, which can then be used for a best guess fix before generating the solver.
    - LLM writes or assembles code when a series of existing backend chunks cannot directly meet the SolverGraph's requirements. The resulting solver, observers, controller, and evidence plan are stored alongside the characteristics of the simulation from the SolverGraph. Eventually, the result itself joins this entry.

- Wall:
    - This one is pretty self-explanatory. In conjunction with the AI I'm using to build and train this tool, I will write a system of checks that can be generalized to anything I try. It will grow as simulations run.
- Live Control (Dr. Strange):
    - Telemetry being fed straight to numerical checks, pattern detectors, trust-horizon models, and independent calculations. The independent calculations will be the hardest to employ while keeping generalizability since the whole idea is to extrapolate to things that can't be calculated. They will help during training.
    - On the ML side: Event-based camera methodology inspired by a talk from Ziyun Wang on Un-EVIMO [13]. The same technique could help categorize background flow and solution-relevant change in an (x,y,t) representation. Good (x,y,t) vs bad (x,y,t) can be trained. More than likely, the chosen ML model to watch a simulation will be part of the ASP.
    - Incident localization connects the anomaly to a plausible cause. We can then give Skuld the right toolbox once it's called to look.
    - "Infinite logging of everything that happens during the sim"
- Judge and Learning:
    - An elaborate concatenation of a primitive library, a Trust Field built from past simulations and any data I can find, an LLM, and the controlled checkpoint system allows for generating a LOT of clean and corrupt trajectories on each run.
    - Clean and corrupted checkpoint trajectories can be used to update the Primitive Atlas and later Judge versions between runs; for held-out demonstrations, the Judge is frozen beforehand and returns a deterministic classification like if it's supported, bounded, unsupported, or rejected. A separate LLM check assesses consistency with the original prompt.
- Automation:
    - I will be making an interface for an LLM to monitor and control the training of this tool according to my wishes, with checkpoints. Important/anomalous iterations and phases will be logged and noted for future presentation and to trace the evolution of the tool.

**Materials and Cost:**

I'm planning around a working budget of $9,000–$14,500 beyond the hardware I already have. Roughly $5,500-$7,500 for a workstation or upgrades, $2,000-$4,000 for model access over the project, and $1,500-$3,000 for storage, cloud compute, or random bursts of HPC runs. Any simulation worthy of an HPC will be more of a benchmark than something I need to train. This could change if I end up creating something quicker and more capable than I'm imagining.

**Execution Plan:**

I've got a plan. I already purchased the PC that will be hosting my model. Up until school starts, I will be working on the harness + IDE + Kubernetes (since I will have four PCs I'm operating across) + vibe coding setup. Once I have created an initial Primitive Atlas and the CFD kernel, this whole thing will run like an engine. I'm going to carefully track storage, monitor logs, create interactive interface from my phone to pause or end or interrupt tasking. Pydantic Logfire style. It's going to be a lot of fun. I designed this project and architecture with automation in mind. If a tool meant to save an engineer some time always needs the engineer to babysit, is it even worth building?

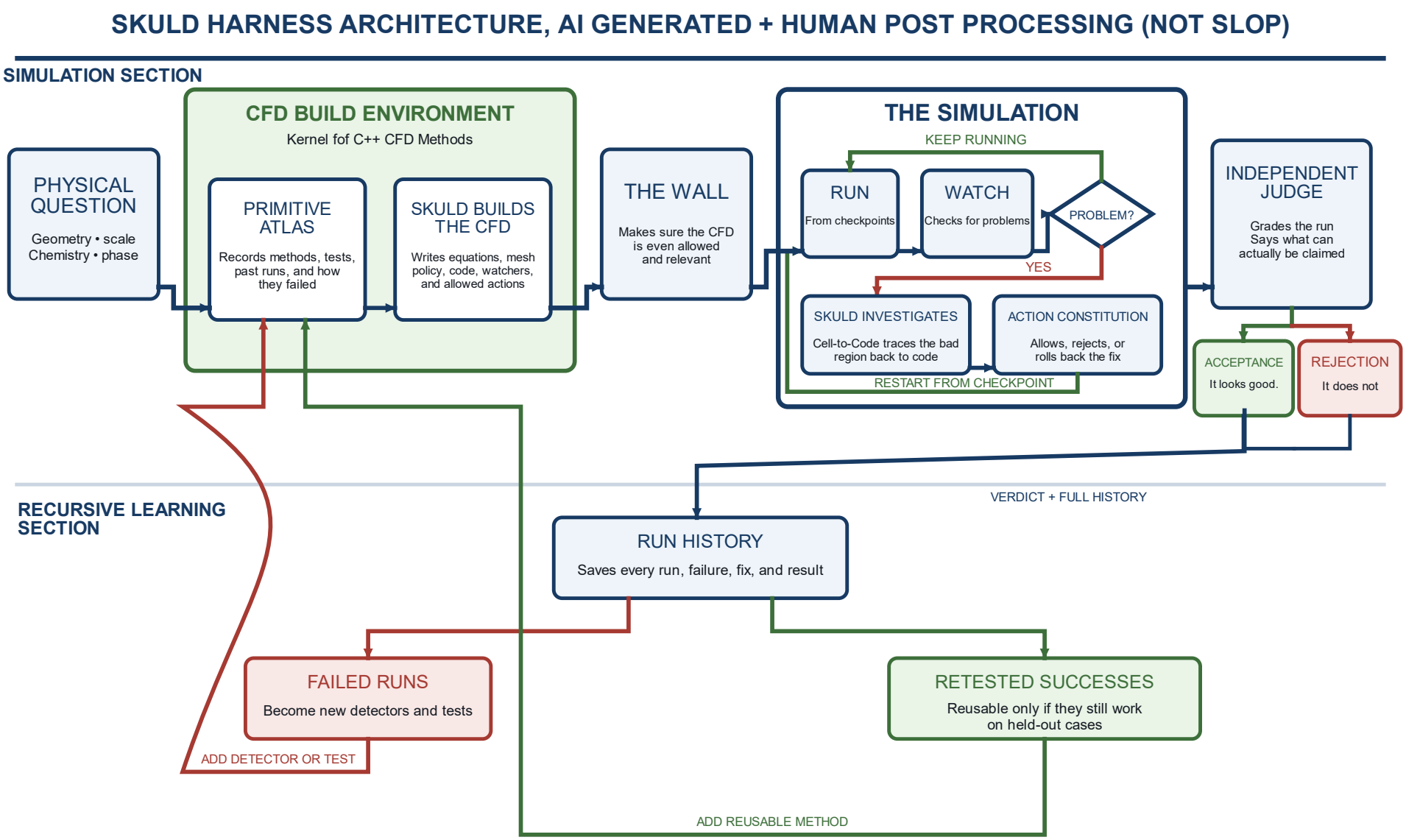



Figure 1. Skuld loop. I was going to make this again myself in Canva but 5.6 Sol just did *such* a good job, it would be a shame not to honour its hard work. Plus, it's an AI-based thesis anyways.

**References:**


[1] Edward A. Luke; Thomas George. *Loci: A Rule-Based Framework for Parallel Multi-Disciplinary Simulation Synthesis.* DOI: 10.1017/S0956796805005514.

[2] Kevin Ellis; Catherine Wong; Maxwell Nye; Mathias Sable-Meyer; Luc Cary; Lucas Morales; Luke Hewitt; Armando Solar-Lezama; Joshua B. Tenenbaum. *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning.* doi:10.1145/3453483.3454080.

[3] Alexander Novikov; Ngân Vũ; Marvin Eisenberger; Emilien Dupont; Po-Sen Huang; Adam Zsolt Wagner; Sergey Shirobokov; Borislav Kozlovskii; Francisco J. R. Ruiz; Abbas Mehrabian; M. Pawan Kumar; Abigail See; Swarat Chaudhuri; George Holland; Alex Davies; Sebastian Nowozin; Pushmeet Kohli; Matej Balog. *AlphaEvolve: A coding agent for scientific and algorithmic discovery.* DOI: 10.48550/arXiv.2506.13131.

[4] E Fan; Kang Hu; Zhuowen Wu; Jiangyang Ge; Jiawei Miao; Yuzhi Zhang; He Sun; Weizong Wang; Tianhan Zhang. *ChatCFD: An LLM-Driven Agent for End-to-End CFD Automation with Structured Knowledge and Reasoning.* DOI: arXiv:2506.02019.

[5] Sandeep Pandey; Ran Xu; Wenkang Wang; Xu Chu. *OpenFOAMGPT: A retrieval-augmented large language model (LLM) agent for OpenFOAM-based computational fluid dynamics.* DOI: 10.1063/5.0257555.

[6] Shanda Li; Tanya Marwah; Junhong Shen; Weiwei Sun; Andrej Risteski; Yiming Yang; Ameet Talwalkar. *CodePDE: An Inference Framework for LLM-Driven PDE Solver Generation.* DOI: arXiv.2505.08783.

[7] Seth Karten; Alex L. Zhang; Kevin Thomas; Sebastian Müller; Elie Bakouch; Daniel Auras; Mika Senghaas; Fares Obeid; Konstantin Dunas; Johannes Hagemann; Sami Jaghouar. *Prime Agent: A Self-Improving RLM Harness.* DOI: 10.48550/arXiv.2608.23552.

[8] Weiqun Zhang; Ann Almgren; Vince Beckner; John Bell; Johannes Blaschke; Cy Chan; Marcus Day; Brian Friesen; Kevin Gott; Daniel Graves; Max Katz; Andrew Myers; Tan Nguyen; Andrew Nonaka; Michele Rosso; Samuel Williams; Michael Zingale. *AMReX: A Framework for Block-Structured Adaptive Mesh Refinement.* DOI: 10.21105/joss.01370.

[9] Marc T. Henry de Frahan; Lucas Esclapez; Jon Rood; Nicholas T. Wimer; Paul Mullowney; Bruce A. Perry; Landon Owen; Hariswaran Sitaraman; Shashank Yellapantula; Malik Hassanaly; Mohammad J. Rahimi; Michael J. Martin; Olga A. Doronina; Sreejith N. A.; Martin Rieth; Wenjun Ge; Ramanan Sankaran; Ann S. Almgren; Weiqun Zhang; John B. Bell; Ray Grout; Marc S. Day; Jacqueline H. Chen. *The Pele Simulation Suite for Reacting Flows at Exascale.* DOI: 10.1137/1.9781611977967.2.

[10] Patrick J. Roache. *Code Verification by the Method of Manufactured Solutions.* DOI: https://doi.org/10.1115/1.1436090.

[11] American Society of Mechanical Engineers. *Standard for Verification and Validation in Computational Fluid Dynamics and Heat Transfer.* DOI: link to the book.

[12] Stéphane Clain; Steven Diot; Raphaël Loubère. *A high-order finite volume method for systems of conservation laws—Multi-dimensional Optimal Order Detection (MOOD).* DOI: 10.1016/j.jcp.2011.02.026.

[13] Ziyun Wang; Jinyuan Guo; Kostas Daniilidis. *Un-EVIMO: Unsupervised Event-Based Independent Motion Segmentation.* DOI: arXiv:2312.00114.

[14] Piotr Wolański; Wojciech Balicki; Wojciech Perkowski; Artur Bilar. *Experimental research of liquid-fueled continuously rotating detonation chamber*.. DOI: 10.1007/s00193-021-01014-w.

[15] Gao Xu; Yu-wen Wu; Chun-sheng Weng; Quan Zheng; Qiao-dong Bai; Hao-long Meng; Chen-wei Ding. *Eulerian-Lagrangian modeling of liquid-fueled rotating detonation with varied equivalence ratios.* DOI: 10.1177/09544100251331684.

[16] Han Peng; Ralf Deiterding. *High-resolution numerical simulation of rotating detonation waves with parallel adaptive mesh refinement.* DOI: 10.1016/j.jaecs.2024.100316.